

Name: Samarth Gaikwad

Roll No: A-20

Practical 1:

Write an X86/64 ALP to accept five 64 bit Hexadecimal numbers from user and store them in an array and display the accepted numbers.

section .data

msg1 db "Enter 5 hexadecimal numbers (64-bit):", 10

len1 equ \$ - msg1

msg2 db "The entered numbers are:", 10

len2 equ \$ - msg2

newline db 10

section .bss

array resq 5 ; array to store 5 numbers (64-bit each)

buffer resb 17 ; buffer for input (16 hex digits + newline)

section .text

global _start

_start:

; Print prompt

mov rax, 1 ; sys_write

```
mov rdi, 1      ; stdout
```

```
mov rsi, msg1
```

```
mov rdx, len1
```

```
syscall
```

```
mov rcx, 5      ; loop counter for 5 inputs
```

```
mov rbx, array  ; pointer to array
```

```
read_loop:
```

```
    ; Read input
```

```
    mov rax, 0      ; sys_read
```

```
    mov rdi, 0      ; stdin
```

```
    mov rsi, buffer
```

```
    mov rdx, 17
```

```
    syscall
```

```
    ; Convert ASCII hex to number
```

```
    mov rsi, buffer
```

```
    xor rax, rax    ; result = 0
```

```
    mov rdx, 16     ; max 16 hex digits
```

```
convert_loop:
```

```
    shl rax, 4      ; multiply result by 16
```

```
    mov bl, [rsi]
```

```
    cmp bl, 10      ; newline?
```

```
    je store_number
```

```
    cmp bl, '0'  
    jl next_digit  
    cmp bl, '9'  
    jle digit_num
```

```
    cmp bl, 'A'  
    jl lower_check  
    cmp bl, 'F'  
    jle digit_upper
```

lower_check:

```
    cmp bl, 'a'  
    jl next_digit  
    cmp bl, 'f'  
    jle digit_lower
```

```
    jmp next_digit
```

digit_num:

```
    sub bl, '0'  
    jmp add_digit
```

digit_upper:

```
    sub bl, 'A'  
    add bl, 10  
    jmp add_digit
```

digit_lower:

sub bl, 'a'

add bl, 10

add_digit:

add al, bl

next_digit:

inc rsi

dec rdx

jnz convert_loop

store_number:

mov [rbx], rax ; store in array

add rbx, 8 ; next element

loop read_loop

; Print message

mov rax, 1

mov rdi, 1

mov rsi, msg2

mov rdx, len2

syscall

; Display numbers

```
mov rcx, 5
mov rbx, array
```

```
print_loop:
```

```
mov rax, [rbx]
```

```
; Convert number to ASCII hex
```

```
mov rdi, buffer
```

```
mov rdx, 16
```

```
hex_loop:
```

```
mov r8, rax
```

```
and r8, 0Fh
```

```
cmp r8, 9
```

```
jle hex_digit
```

```
add r8, 7
```

```
hex_digit:
```

```
add r8, '0'
```

```
mov [rdi + rdx - 1], r8b
```

```
shr rax, 4
```

```
dec rdx
```

```
jnz hex_loop
```

```
; Print number
```

```
mov rax, 1
mov rdi, 1
mov rsi, buffer
mov rdx, 16
syscall
```

```
; Print newline
mov rax, 1
mov rdi, 1
mov rsi, newline
mov rdx, 1
syscall
```

```
add rbx, 8
loop print_loop
```

```
; Exit
mov rax, 60
xor rdi, rdi
syscall
```

Output:

The entered numbers are:

1A2B3C4D5E6F7788

FFFFFFFFFFFFFFFF

123456789ABCDEF0

ABCDEF12345678

Practical No: 2

Write an X86/64 ALP to accept a string and to display its length.

section .data

msg1 db "Enter a string: ", 0

len1 equ \$ - msg1

msg2 db "Length of string is: ", 0

len2 equ \$ - msg2

newline db 10

section .bss

buffer resb 100 ; input buffer

result resb 3 ; to store length (max 2 digits + null)

section .text

global _start

_start:

; Print prompt

mov rax, 1 ; sys_write

mov rdi, 1

mov rsi, msg1

```
mov rdx, len1
```

```
syscall
```

```
; Read string
```

```
mov rax, 0 ; sys_read
```

```
mov rdi, 0
```

```
mov rsi, buffer
```

```
mov rdx, 100
```

```
syscall
```

```
mov rcx, rax ; rcx = number of bytes read
```

```
dec rcx ; exclude newline
```

```
; Print message
```

```
mov rax, 1
```

```
mov rdi, 1
```

```
mov rsi, msg2
```

```
mov rdx, len2
```

```
syscall
```

```
; Convert length (rcx) to ASCII
```

```
mov rax, rcx
```

```
mov rbx, 10
```

```
mov rdi, result + 2 ; pointer to end
```

```
mov byte [rdi], 0
```

```
convert:
```

```

dec rdi
xor rdx, rdx
div rbx      ; rax / 10
add dl, '0'
mov [rdi], dl
test rax, rax
jnz convert

; Print length
mov rax, 1
mov rdi, 1
mov rsi, rdi ; (fix pointer below)
mov rsi, rdi ; placeholder

; correct pointer
mov rsi, rdi

; actually rdi was changed, so reload pointer
lea rsi, [result]
mov rdx, 3
syscall

; newline
mov rax, 1
mov rdi, 1
mov rsi, newline
mov rdx, 1

```

```
syscall
```

```
; exit
```

```
mov rax, 60
```

```
xor rdi, rdi
```

```
syscall
```

Output:

Enter a string: Hello

Length of string is: 5

Enter a string: Assembly Language

Length of string is: 17

Practical No: 3

Write an X86/64 ALP to count number of positive and negative numbers from the array.

section .data

; Sample array (you can modify values)

array dq 10, -5, 20, -15, 30, -25

count equ 6

msg1 db "Positive numbers: ", 0

len1 equ \$ - msg1

msg2 db "Negative numbers: ", 0

len2 equ \$ - msg2

newline db 10

section .bss

pos resb 3 ; to store positive count (ASCII)

neg resb 3 ; to store negative count (ASCII)

section .text

global _start

_start:

```

mov rcx, count    ; loop counter
mov rsi, array    ; pointer to array

xor r8, r8        ; positive count = 0
xor r9, r9        ; negative count = 0

check_loop:
    mov rax, [rsi]

    cmp rax, 0
    jl negative

    inc r8         ; positive
    jmp next

negative:
    inc r9         ; negative

next:
    add rsi, 8
    loop check_loop

; ----- Print Positive Count -----
mov rax, 1
mov rdi, 1
mov rsi, msg1
mov rdx, len1

```

```
syscall
```

```
mov rax, r8
```

```
call print_num
```

```
; newline
```

```
mov rax, 1
```

```
mov rdi, 1
```

```
mov rsi, newline
```

```
mov rdx, 1
```

```
syscall
```

```
; ----- Print Negative Count -----
```

```
mov rax, 1
```

```
mov rdi, 1
```

```
mov rsi, msg2
```

```
mov rdx, len2
```

```
syscall
```

```
mov rax, r9
```

```
call print_num
```

```
; newline
```

```
mov rax, 1
```

```
mov rdi, 1
```

```
mov rsi, newline
```

```
mov rdx, 1
```

syscall

; exit

mov rax, 60

xor rdi, rdi

syscall

; ----- Procedure: print_num -----

; Input: number in RAX

print_num:

mov rbx, 10

mov rcx, 0

mov rdi, pos + 2 ; reuse buffer

convert_loop:

xor rdx, rdx

div rbx

add dl, '0'

dec rdi

mov [rdi], dl

inc rcx

test rax, rax

jnz convert_loop

; print number

mov rax, 1

```
mov rsi, rdi
mov rdx, rcx
mov rdi, 1
syscall

ret
```

Output:

10, -5, 20, -15, 30, -25

Positive numbers: 3

Negative numbers: 3

Practical No: 4

Write X86/64 ALP to perform non-overlapped block transfer without string specific instructions.
Block containing data can be defined in the data segment.

```
section .data
```

```
src dq 11, 22, 33, 44, 55    ; source block
```

```
count equ 5
```

```
section .bss
```

```
dest resq 5                  ; destination block
```

```
section .text
```

```
global _start
```

```
_start:
```

```
    mov rcx, count    ; number of elements
```

```
    mov rsi, src       ; source pointer
```

```
    mov rdi, dest      ; destination pointer
```

```
copy_loop:
```

```
    mov rax, [rsi]     ; load from source
```

```
    mov [rdi], rax     ; store to destination
```

```
    add rsi, 8          ; move to next (64-bit = 8 bytes)
```

```
add rdi, 8
```

```
loop copy_loop
```

```
; ---- Exit ----
```

```
mov rax, 60
```

```
xor rdi, rdi
```

```
syscall
```

Output:

Before execution

11 22 33 44 55

After Execution

11 22 33 44 55

Practical No: 5

Write X86/64 ALP to perform overlapped block transfer with string specific instructions Block containing data can be defined in the data segment.

```
section .data
```

```
    block db 10, 20, 30, 40, 50, 60, 70, 80
```

```
    len equ 8
```

```
section .text
```

```
    global _start
```

```
_start:
```

```
    ; Example:
```

```
    ; Copy last 5 bytes to position starting at index 2
```

```
    ; (overlapping case)
```

```
    mov rcx, 5          ; number of bytes to copy
```

```
    lea rsi, [block + 4] ; source end (last byte of source)
```

```
    lea rdi, [block + 6] ; destination end
```

```
    std                ; set direction flag (backward)
```

```
    rep movsb          ; copy bytes
```

```
cld          ; clear direction flag (good practice)
```

```
; ----- Exit -----
```

```
mov rax, 60
```

```
xor rdi, rdi
```

```
syscall
```

Output:

Initial Block

10 20 30 40 50 60 70 80

Final Block

10 20 30 40 50 60 50 60

Practical No:6

Write X86/64 ALP to perform multiplication of two 8-bit hexadecimal numbers. Use successive addition method only.

section .data

num1 db 05h ; first 8-bit number

num2 db 03h ; second 8-bit number

msg db "Result: ", 0

len equ \$ - msg

newline db 10

section .bss

result resb 4 ; buffer to print result (hex)

section .text

global _start

_start:

mov al, [num1] ; AL = first number

mov bl, [num2] ; BL = second number

xor ax, ax ; AX = 0 (result)

```
mov cl, bl      ; loop counter = num2
```

multiply:

```
add ax, [num1]  ; AX = AX + num1
```

```
dec cl
```

```
jnz multiply
```

```
; ---- Print message ----
```

```
mov rax, 1
```

```
mov rdi, 1
```

```
mov rsi, msg
```

```
mov rdx, len
```

```
syscall
```

```
; ---- Convert result (AX) to HEX ----
```

```
mov bx, ax
```

```
mov rdi, result + 3
```

```
mov rcx, 4
```

hex_convert:

```
mov dl, bl
```

```
and dl, 0Fh
```

```
cmp dl, 9
```

```
jbe digit
```

```
add dl, 7
```

digit:

add dl, '0'

mov [rdi], dl

shr bx, 4

dec rdi

loop hex_convert

; ---- Print result ----

mov rax, 1

mov rdi, 1

mov rsi, result

mov rdx, 4

syscall

; newline

mov rax, 1

mov rdi, 1

mov rsi, newline

mov rdx, 1

syscall

; ----- Exit -----

mov rax, 60

xor rdi, rdi

syscall

Output:

num1 = 05h

num2 = 03h

Result: 000F

Practical No: 7

Write an X86/64 ALP to find the largest of given Byte/Word/Dword/64-bit numbers.

section .data

array dq 10, 50, 25, 90, 60 ; 64-bit numbers

count equ 5

msg db "Largest number: ", 0

len equ \$ - msg

newline db 10

section .bss

result resb 16 ; buffer for printing hex

section .text

global _start

_start:

mov rsi, array

mov rcx, count

mov rax, [rsi] ; assume first element is largest

add rsi, 8

```
dec rcx
```

```
find_loop:
```

```
mov rbx, [rsi]
```

```
cmp rbx, rax
```

```
jle skip
```

```
mov rax, rbx      ; update largest
```

```
skip:
```

```
add rsi, 8
```

```
loop find_loop
```

```
; ---- Print message ----
```

```
mov rdi, 1
```

```
mov rax, 1
```

```
mov rsi, msg
```

```
mov rdx, len
```

```
syscall
```

```
; ---- Convert RAX to HEX ----
```

```
mov rbx, rax
```

```
mov rdi, result + 15
```

```
mov rcx, 16
```

```
convert:
```

```
mov dl, bl
and dl, 0Fh
```

```
cmp dl, 9
jbe digit
add dl, 7
```

digit:

```
add dl, '0'
mov [rdi], dl
```

```
shr rbx, 4
dec rdi
loop convert
```

```
; ---- Print result ----
```

```
mov rax, 1
mov rdi, 1
mov rsi, result
mov rdx, 16
syscall
```

```
; newline
mov rax, 1
mov rdi, 1
mov rsi, newline
mov rdx, 1
```

```
syscall
```

```
; ---- Exit ----
```

```
mov rax, 60
```

```
xor rdi, rdi
```

```
syscall
```

Output:

10, 50, 25, 90, 60

Largest number: 0000000000000005A

Practical No: 8

Write X86/64 ALP to convert 4-digit Hex number into its equivalent BCD number.

section .data

hexnum dw 1234h ; 4-digit hex number

msg db "BCD Equivalent: ", 0

len equ \$ - msg

newline db 10

section .bss

bcd resb 5 ; max 5 decimal digits

section .text

global _start

_start:

mov ax, [hexnum] ; load hex number

mov bx, 10 ; divisor = 10

mov rdi, bcd + 4 ; pointer to end

mov rcx, 5 ; max digits

convert:

```

xor dx, dx          ; clear remainder
div bx              ; AX / 10

add dl, '0'         ; convert remainder to ASCII
mov [rdi], dl

dec rdi
loop convert

; ---- Print message ----
mov rax, 1
mov rdi, 1
mov rsi, msg
mov rdx, len
syscall

; ---- Print BCD result ----
mov rax, 1
mov rdi, 1
mov rsi, bcd
mov rdx, 5
syscall

; newline
mov rax, 1
mov rdi, 1
mov rsi, newline

```

```
mov rdx, 1  
syscall
```

```
; ----- Exit -----  
mov rax, 60  
xor rdi, rdi  
syscall
```

Output:

hexnum = 1234h

BCD Equivalent: 04660

Practical No: 9

Write X86/64 ALP to convert 5- digit BCD number into its equivalent HEX number.

section .data

bcd db 01h,02h,03h,04h,05h ; BCD = 12345

len equ 5

msg db "HEX Equivalent: "

msglen equ \$-msg

section .bss

result resq 1

buffer resb 16 ; for ASCII output

section .text

global _start

_start:

mov rsi, bcd

mov rcx, len

xor rax, rax ; result = 0

convert:

mov bl, [rsi]

and bl, 0Fh

```

; rax = rax * 10
mov rdx, rax
shl rax, 3      ; *8
shl rdx, 1      ; *2
add rax, rdx    ; *10

add rax, rbx    ; + digit

inc rsi

loop convert

mov [result], rax

; ----- PRINT MESSAGE -----
mov rax, 1
mov rdi, 1
mov rsi, msg
mov rdx, msglen
syscall

; ----- CONVERT TO HEX STRING -----
mov rax, [result]
mov rbx, 16
mov rdi, buffer
add rdi, 15
mov byte [rdi], 0

```

hex_convert:

dec rdi

xor rdx, rdx

div rbx ; divide by 16

cmp dl, 9

jbe digit

add dl, 7 ; A-F adjust

digit:

add dl, '0'

mov [rdi], dl

test rax, rax

jnz hex_convert

; ----- PRINT RESULT -----

mov rax, 1

mov rdi, 1

mov rsi, rdi

mov rsi, rdi

mov rsi, rdi

; correct pointer

mov rsi, rdi

; (fix: actual start is rdi after loop)

```
mov rsi, rdi
```

```
; print string
```

```
mov rdx, buffer + 15
```

```
sub rdx, rdi
```

```
mov rax, 1
```

```
mov rdi, 1
```

```
mov rsi, rdi
```

```
syscall
```

```
; exit
```

```
mov rax, 60
```

```
xor rdi, rdi
```

```
syscall
```

Output

HEX Equivalent: 3039